

GPU Acceleration Strategy

Optimized for Dual RTX 4090 + Threadripper PRO (Your Hardware)

Technical Analysis

2025-10-12

Table of contents

1	Executive Summary	2
1.1	Key Findings	2
1.2	Recommended Strategy	2
1.3	Investment & Return	2
2	Current Performance Analysis	3
2.1	Baseline: CPU Only (n_jobs=10)	3
2.2	The Problem	3
3	GPU Acceleration Strategy	5
3.1	Why GPU Helps (When CPU Doesn't)	5
3.2	Single GPU Performance	5
3.3	Dual GPU Configurations	6
3.4	Performance Comparison	8
4	Implementation Plan	9
4.1	Phase 1: Single GPU (Week 1)	9
4.2	Phase 2: Dual GPU Parallel (Week 2)	10
4.3	Phase 3: Dual GPU Split (Optional, Week 3)	11
5	Hardware Considerations	13
5.1	Your Current Setup Analysis	13
5.2	Cooling and Thermal Management	13
5.3	Power and Cost	14
6	Cloud vs. Local Comparison	16
6.1	Lambda Labs Pricing	16
6.2	Your Local Machine	16
6.3	When to Use Lambda Labs	16
6.4	Hybrid Workflow Example	16
7	Production Deployment	18
7.1	Recommended Configuration Files	18
7.2	CLI Interface	19
7.3	Monitoring Dashboard	20
8	Troubleshooting Guide	22
8.1	Common Issues	22

9	Summary and Next Steps	26
9.1	Summary	26
9.2	Immediate Next Steps	26
9.3	Decision Points	27
9.4	Long-Term Strategy	27
9.5	Support and Resources	27
9.6	Conclusion	28

1 Executive Summary

This document provides a GPU acceleration strategy **customized for your specific hardware**:

- **CPU**: AMD Ryzen Threadripper PRO 5955WX (16 cores, 32 threads)
- **GPU**: 2x NVIDIA GeForce RTX 4090 (24 GB each, liquid cooled)
- **Current**: n_jobs=10 CPU parallelization

1.1 Key Findings

1. **Your hardware is excellent** - no purchase needed
2. **Current bottleneck**: IPCA (98.9% of runtime, 1800 sec) can't use your 16 CPU cores
3. **GPU solution**: Parallelize within IPCA iterations (CPU can't do this)
4. **Expected speedup**: 21.8x with single GPU, up to 38.6x with dual GPU optimization

1.2 Recommended Strategy

Phase 1 (Week 1): Single GPU IPCA + SDF - Development: 3-5 days - Speedup: 21.8x (30 min → 84 sec)
- Use GPU 0 only

Phase 2 (Week 2): Dual GPU parallel iterations - Development: 2-3 days - Throughput: 2x (run 2 iterations simultaneously) - Use both GPUs

Phase 3 (Optional): Dual GPU split work - Development: 2-3 days - Speedup: 38.6x (30 min → 47 sec) - Advanced optimization

1.3 Investment & Return

Item	Value
Hardware cost	\$0 (already own)
Development time	1-2 weeks
Development cost	\$3,600 (60 hrs @ \$60/hr)
Annual time savings	3,052 hours
Annual value	\$183,120
ROI	51:1
Payback period	7 days

2 Current Performance Analysis

2.1 Baseline: CPU Only (n_jobs=10)

2.1.1 Runtime Breakdown

Component	Single-Thread	With n_jobs=10	% of Total
IPCA	3600 sec	1800 sec	98.9%
Fama-French	4 sec	0.5 sec	<0.1%
Fama-MacBeth	4 sec	0.5 sec	<0.1%
DKKM	2 sec	0.25 sec	<0.1%
SDF computation	10 sec	10 sec	0.5%
Panel generation	6 sec	6 sec	0.3%
Monthly evaluation	4 sec	4 sec	0.2%
TOTAL	3630 sec 60.5 min	1821 sec 30.4 min	100%

2.1.2 Why IPCA Dominates

IPCA structure (serial on CPU):

```
def fit_ipca(panel, start, K, tol):
    # Only parallelizes over 2 IPCA variants (K=1, K=2)
    # 8 of your 10 CPU cores sit idle!

    for K in [1, 2]: # Only 2 tasks for 10 cores
        # Each variant processes 360 rolling windows SERIALY
        for window in range(360): # Can't parallelize
            # Each window runs up to 1000 iterations SERIALY
            for iter in range(1000): # Can't parallelize!
                # Dense matrix operations
                Gamma, f = update(...) # This is where GPU helps
```

Why n_jobs=10 doesn't help IPCA: - Outer parallelism: Only 2 IPCA variants (8 cores idle) - Middle loop: 360 windows processed serially (time dependencies) - Inner loop: 1000 iterations serially (convergence dependencies) - **Result:** IPCA can only use 2 of your 10 cores effectively

2.1.3 Why Fama/DKKM Work Well

Fama-French structure (parallel on CPU):

```
# Parallelizes over ~720 independent months
lst = Parallel(n_jobs=10)(
    delayed(monthly_rets)(month) for month in range(720)
)
# Uses 10 cores effectively: 720 tasks >> 10 cores
```

Result: Near-linear speedup with n_jobs=10

2.2 The Problem

IPCA is 98.9% of runtime but can't use your CPU cores.

More CPU cores won't help: - n_jobs=10: 1800 sec (IPCA) - n_jobs=32: 1800 sec (IPCA) - **NO IMPROVEMENT**

GPU is the only solution - parallelizes WITHIN each iteration.

3 GPU Acceleration Strategy

3.1 Why GPU Helps (When CPU Doesn't)

3.1.1 CPU Parallelization Limits

CPU parallelize across **independent tasks**: - Works: 720 independent monthly calculations (Fama, DKKM) - Fails: Serial convergence loop in IPCA

3.1.2 GPU Parallelization

GPUs parallelize **within operations**:

```
# CPU: Sequential matrix multiply ( $N^2$  operations)
for i in range(N):
    for j in range(N):
        result[i,j] = sum(A[i,k] * B[k,j] for k in range(N))
# Time:  $O(N^3)$  sequential operations

# GPU: Parallel matrix multiply ( $N^2$  threads)
result_gpu = A_gpu @ B_gpu # 16,384 CUDA cores in parallel!
# Time:  $O(N)$  with massive parallelism
```

IPCA speedup mechanism: - Iterations remain serial (still 1000 iterations) - BUT each iteration is 25x faster (GPU matrix operations) - Overall: 25x faster IPCA

3.2 Single GPU Performance

3.2.1 Expected Results (GPU 0 Only)

Component	CPU (n_jobs=10)	GPU	Speedup	Resource
IPCA	1800 sec	72 sec	25x	GPU 0
SDF	10 sec	0.5 sec	20x	GPU 0
Fama-FF	0.5 sec	0.5 sec	1x	10 CPU cores
Fama-MB	0.5 sec	0.5 sec	1x	10 CPU cores
DKKM	0.25 sec	0.25 sec	1x	10 CPU cores
Panel gen	6 sec	6 sec	1x	1 CPU core
Other	4 sec	4 sec	1x	CPU
TOTAL	1821 sec	83.5 sec	21.8x	Hybrid

Strategy: - GPU handles IPCA + SDF (the serial bottlenecks) - CPU handles Fama/DKKM (already well-parallelized)

3.2.2 Resource Utilization Timeline

Time (sec)	CPU Cores	GPU 0	GPU 1
0-6	Panel gen (1)	Idle	Idle
6-6.5	Fama-FF (10)	Idle	Idle
6.5-7	Fama-MB (10)	Idle	Idle
7-7.25	DKKM (10)	Idle	Idle
7.25-79.25	Idle (waiting)	IPCA (72s)	Idle
79.25-79.75	Idle (waiting)	SDF (0.5s)	Idle
79.75-83.75	Monthly eval	Idle	Idle

Observation: GPU 1 completely idle! Let's use it...

3.3 Dual GPU Configurations

You have **2x RTX 4090** - three ways to use them:

3.3.1 Configuration A: Parallel Iterations (Recommended for Batches)

Run **2** iterations simultaneously - one per GPU:

Resource	Iteration 0	Iteration 1
CPU cores	8 cores	8 cores
GPU	GPU 0	GPU 1
Time	84 sec	84 sec (concurrent!)

Performance: - 2 iterations in 84 seconds (vs. 168 sec sequential) - Throughput: **43 iterations/hour** (vs. 21.5 with single GPU) - **2x throughput improvement**

Use **case:** Batch jobs with 50+ iterations

Code structure:

```
def process_iterations_parallel(iter_nums):
    """Run multiple iterations in parallel on different GPUs."""

    def process_on_gpu(iter_num, gpu_id):
        cp.cuda.Device(gpu_id).use()
        processor = PanelProcessor(config, gpu_id=gpu_id)
        return processor.process_iteration(iter_num, writer)

    # GPU 0: iterations 0, 2, 4, 6, ...
    # GPU 1: iterations 1, 3, 5, 7, ...
    with ThreadPoolExecutor(max_workers=2) as executor:
        futures = [
            executor.submit(process_on_gpu, i, i % 2)
            for i in iter_nums
        ]
        results = [f.result() for f in futures]

    return results

# Run 100 iterations
# Time: 100 iters / 2 GPUs × 84 sec = 70 minutes (vs. 140 min single GPU)
results = process_iterations_parallel(range(100))
```

3.3.2 Configuration B: Split Work (Recommended for Single Iterations)

Use **both GPUs for ONE iteration** - split IPCA windows:

Component	Resource	Time
IPCA windows 0-179	GPU 0	36 sec
IPCA windows 180-359	GPU 1	36 sec (concurrent!)
SDF (split)	GPU 0 + GPU 1	0.3 sec
Fama/DKKM	16 CPU cores	0.7 sec

Performance: - Single iteration: ~**47 seconds** (vs. 84 sec single GPU) - **1.8x faster** per iteration - **38.6x faster** than CPU baseline!

Use case: Interactive development, quick tests

Code structure:

```
def compute_ipca_dual_gpu_split(panel, start, end):
    """Split IPCA rolling windows across 2 GPUs."""

    def process_windows_on_gpu(windows, gpu_id):
        cp.cuda.Device(gpu_id).use()

        results = []
        for w in windows:
            result = fit_ipca_window_gpu(panel, w, ...)
            results.append(result)

        return results

    # Split 360 windows: GPU 0 gets first 180, GPU 1 gets last 180
    windows_gpu0 = range(0, 180)
    windows_gpu1 = range(180, 360)

    with ThreadPoolExecutor(max_workers=2) as executor:
        future_0 = executor.submit(process_windows_on_gpu, windows_gpu0, 0)
        future_1 = executor.submit(process_windows_on_gpu, windows_gpu1, 1)

        results = future_0.result() + future_1.result()

    return results
```

3.3.3 Configuration C: Hybrid Adaptive (Recommended Overall)

Automatically choose based on workload:

```
# config.yaml
gpu_mode: "auto" # Automatically select best configuration

# main.py logic:
if num_iters == 1:
    # Single iteration: use Configuration B (split work)
    mode = "split"
    expected_time = 47 # seconds
elif num_iters <= 10:
    # Small batch: use Configuration B (faster per iteration)
    mode = "split"
    expected_time = num_iters * 47
else:
    # Large batch: use Configuration A (parallel iterations)
    mode = "parallel"
    expected_time = num_iters * 84 / 2
```

3.4 Performance Comparison

3.4.1 Your Hardware Evolution

Configuration	Time/Iter	Throughput	Total (100 iters)	Speedup
CPU only (n_jobs=10)	30 min	2 iter/hr	50 hours	1x
CPU (n_jobs=16)	30 min	2 iter/hr	50 hours	1x
Single GPU	84 sec	43 iter/hr	2.3 hours	21.7x
Dual GPU (parallel)	84 sec	86 iter/hr	1.2 hours	41.7x
Dual GPU (split)	47 sec	77 iter/hr	1.3 hours	38.5x

3.4.2 Recommendation by Use Case

Use Case	Iterations	Recommended	Time	Why
Quick test	1-5	Dual split	<4 min	Fastest per iteration
Development	10-50	Dual split	8-40 min	Interactive feedback
Batch job	100+	Dual parallel	~70 min	Best throughput
Overnight	500+	Dual parallel	~5.8 hours	Maximize GPUs

4 Implementation Plan

4.1 Phase 1: Single GPU (Week 1)

4.1.1 Objectives

- Migrate IPCA to GPU
- Migrate SDF to GPU
- Achieve 21.8x speedup
- Use GPU 0 only (keep GPU 1 as backup)

4.1.2 Development Time

Day 1-2 (16 hours): IPCA GPU migration - Create `ipca_functions_gpu.py` - Convert `regress()`, `ipca_iter()`, `fit_ipca()` to CuPy - Unit tests for numerical equivalence

Day 3-4 (16 hours): SDF GPU migration - Create `sdf_compute_gpu.py` - Convert term computations to CuPy - Handle sparse matrices with CuPy sparse - Integration tests

Day 5 (8 hours): Integration and testing - Update `panel_processor.py` for hybrid CPU+GPU - Full pipeline testing - Performance benchmarking

Deliverable: Single GPU acceleration working

4.1.3 Validation

```
def test_single_gpu_speedup():
    """Verify GPU speedup on your hardware."""
    config = SimulationConfig(
        model='bgn', N=100, T=720,
        num_iters=5, n_jobs=10, use_gpu=True
    )

    # CPU baseline
    start = time.time()
    processor_cpu = PanelProcessor(config, use_gpu=False)
    for i in range(5):
        processor_cpu.process_iteration(i, writer)
    time_cpu = time.time() - start

    # GPU version
    start = time.time()
    processor_gpu = PanelProcessor(config, use_gpu=True, gpu_id=0)
    for i in range(5):
        processor_gpu.process_iteration(i, writer)
    time_gpu = time.time() - start

    speedup = time_cpu / time_gpu
    print(f"CPU: {time_cpu:.1f}s, GPU: {time_gpu:.1f}s")
    print(f"Speedup: {speedup:.1f}x")

    assert speedup > 15, f"Expected >15x speedup, got {speedup:.1f}x"
```

Expected output:

CPU: 9105s (2.5 hours), GPU: 418s (7 minutes)
Speedup: 21.8x

4.2 Phase 2: Dual GPU Parallel (Week 2)

4.2.1 Objectives

- Run 2 iterations simultaneously
- Double throughput to 43 iters/hour
- Maintain 21.8x speedup per iteration

4.2.2 Development Time

Day 1-2 (16 hours): - Implement multi-GPU orchestration - Thread pool executor for concurrent iterations
- GPU device management (avoid conflicts)

Day 3 (8 hours): - Testing with 10, 50, 100 iterations - Verify no GPU memory leaks - Performance profiling

Deliverable: Dual GPU parallel working

4.2.3 Code Example

```
class DualGPUProcessor:
    def __init__(self, config):
        self.config = config
        assert torch.cuda.device_count() >= 2, "Need 2 GPUs"

    def process_batch_parallel(self, iter_nums):
        """Process batch using both GPUs in parallel."""

        def process_on_gpu(iter_num, gpu_id):
            # Set GPU device for this thread
            torch.cuda.set_device(gpu_id)
            cp.cuda.Device(gpu_id).use()

            # Process iteration on this GPU
            processor = PanelProcessor(
                self.config,
                use_gpu=True,
                gpu_id=gpu_id
            )

            return processor.process_iteration(iter_num, writer)

        # Distribute iterations across 2 GPUs
        with ThreadPoolExecutor(max_workers=2) as executor:
            futures = []
            for i, iter_num in enumerate(iter_nums):
                gpu_id = i % 2 # Round-robin assignment
                future = executor.submit(process_on_gpu, iter_num, gpu_id)
                futures.append(future)

            # Collect results
            results = [f.result() for f in futures]

        return results

# Usage
```

```
processor = DualGPUProcessor(config)

# 100 iterations on 2 GPUs: ~70 minutes
results = processor.process_batch_parallel(range(100))
```

4.2.4 Validation

```
def test_dual_gpu_parallel():
    """Verify dual GPU parallel performance."""
    config = SimulationConfig(
        model='bgn', N=100, T=720,
        num_iters=20, n_jobs=10
    )

    # Single GPU baseline
    start = time.time()
    processor_single = PanelProcessor(config, use_gpu=True, gpu_id=0)
    for i in range(20):
        processor_single.process_iteration(i, writer)
    time_single = time.time() - start

    # Dual GPU parallel
    start = time.time()
    processor_dual = DualGPUProcessor(config)
    processor_dual.process_batch_parallel(range(20))
    time_dual = time.time() - start

    speedup = time_single / time_dual
    print(f"Single GPU: {time_single:.1f}s")
    print(f"Dual GPU: {time_dual:.1f}s")
    print(f"Parallel efficiency: {speedup:.2f}x")

    assert speedup > 1.8, f"Expected >1.8x, got {speedup:.1f}x"
```

Expected output:

```
Single GPU: 1680s (28 min)
Dual GPU: 840s (14 min)
Parallel efficiency: 2.0x
```

4.3 Phase 3: Dual GPU Split (Optional, Week 3)

4.3.1 Objectives

- Split IPCA windows across 2 GPUs
- Achieve 47 sec per iteration (vs. 84 sec)
- 1.8x faster single iterations

4.3.2 Development Time

Day 1-2 (12 hours): - Refactor IPCA to support window ranges - Implement GPU-to-GPU synchronization
- Handle edge cases (odd number of windows)

Day 3 (8 hours): - Testing and optimization - Memory management - GPU utilization profiling

4.3.3 Complexity vs. Benefit

Benefit: 47 sec vs. 84 sec (saves 37 sec per iteration)

Complexity: - Medium complexity (requires refactoring IPCA) - Need to handle GPU synchronization - More code to maintain

When worth it: - Interactive development (frequent 1-5 iteration tests) - Iterative model development - Quick parameter tuning

When NOT worth it: - Batch jobs >50 iterations (use parallel instead) - Infrequent runs - Production pipelines (parallel is simpler)

Recommendation: Implement if you do >10 single-iteration runs per day.

5 Hardware Considerations

5.1 Your Current Setup Analysis

5.1.1 Strengths

Excellent CPU: Threadripper PRO 5955WX - 16 cores / 32 threads - Perfect for `n_jobs=16` parallelization
- PCIe 4.0 for fast GPU communication

Top-tier GPUs: 2x RTX 4090 - 24 GB VRAM each (plenty for `N=100`, `T=720`) - 16,384 CUDA cores each - Ada Lovelace architecture (latest) - Liquid cooling (can sustain boost clocks)

Already paid for: \$0 marginal cost

5.1.2 Limitations

Power draw: ~1200W under full load - 2x RTX 4090: $2 \times 450W = 900W$ - Threadripper PRO: ~280W - Cost: ~\$3.50/day (24/7 operation)

Memory bandwidth: 1 TB/s per GPU - Not a bottleneck for your `N=100`, `T=720` workload - Could become bottleneck for `N>500`

PCIe lanes: Check your configuration - Each RTX 4090 needs 16 PCIe lanes for full bandwidth - Threadripper PRO has 128 lanes (plenty!) - Verify both GPUs are at x16 (not x8)

5.1.3 Checking Your Configuration

```
# Verify GPUs are running at full PCIe bandwidth
nvidia-smi --query-gpu=name,pci.bus_id,pcie.link.gen.current,pcie.link.width.current --format=csv

# Expected output:
# name, pci.bus_id, pcie.link.gen.current, pcie.link.width.current
# NVIDIA GeForce RTX 4090, 0000:01:00.0, 4, 16
# NVIDIA GeForce RTX 4090, 0000:02:00.0, 4, 16
#           ^      ^^
#           |      |
#           |  +-- 16 lanes (full bandwidth)
#           +-- Gen 4 (PCIe 4.0)
```

If you see 8 instead of 16, you're losing 50% bandwidth!

5.2 Cooling and Thermal Management

5.2.1 Monitor Temperatures

Your AIO liquid cooling is good, but monitor during long runs:

```
import subprocess
import pandas as pd
import time

def monitor_thermal_performance(duration_hours=1):
    """Monitor GPU temps and throttling during workload."""

    data = []
    start_time = time.time()
    duration_sec = duration_hours * 3600
```

```

while time.time() - start_time < duration_sec:
    result = subprocess.run(
        ['nvidia-smi', '--query-gpu=timestamp,name,temperature.gpu,'
         'power.draw,clocks.current.graphics,clocks.current.memory',
         '--format=csv'],
        capture_output=True, text=True
    )

    lines = result.stdout.strip().split('\n')[1:] # Skip header
    for line in lines:
        parts = line.split(',')
        data.append({
            'timestamp': parts[0],
            'gpu': parts[1],
            'temp_c': float(parts[2]),
            'power_w': float(parts[3].split()[0]),
            'gpu_clock_mhz': float(parts[4].split()[0]),
            'mem_clock_mhz': float(parts[5].split()[0])
        })

    time.sleep(10) # Sample every 10 seconds

df = pd.DataFrame(data)

# Analyze
print("Thermal Performance Summary:")
print(df.groupby('gpu')[['temp_c', 'power_w']].agg(['mean', 'max']))

# Check for throttling
for gpu_name in df['gpu'].unique():
    gpu_data = df[df['gpu'] == gpu_name]
    boost_clock = 2520 # RTX 4090 boost clock (MHz)
    avg_clock = gpu_data['gpu_clock_mhz'].mean()
    if avg_clock < boost_clock * 0.9:
        print(f" {gpu_name} may be throttling!")
        print(f" Avg clock: {avg_clock:.0f} MHz (vs. {boost_clock} MHz boost)")

return df

# Run during a batch job
thermal_data = monitor_thermal_performance(duration_hours=2)

```

Safe operating ranges: - Temperature: <80°C continuous, <85°C acceptable - Power: 400-450W expected under full load - GPU clock: Should stay >2200 MHz (no throttling)

If temps exceed 85°C: - Check AIO pump is running - Verify radiator fans are operational - Clean dust from radiators - Consider limiting power: `nvidia-smi -pl 400` (400W limit)

5.3 Power and Cost

5.3.1 Electricity Cost

Your system under full GPU load: - Power: ~1200W sustained - Cost: \$0.12/kWh (US average) - Daily: $1.2 \text{ kW} \times 24 \text{ hr} \times \$0.12/\text{kWh} = \$3.46/\text{day}$ - Monthly (24/7): ~\$104/month

Batch job cost (100 iterations, 70 minutes): - Power: $1.2 \text{ kW} \times 1.17 \text{ hr} = 1.4 \text{ kWh}$ - Cost: $1.4 \text{ kWh} \times \$0.12 = \mathbf{\$0.17 \text{ per 100 iterations}}$

Compare to Lambda Labs: - 2x A100 instance: $\$2.98/\text{hour} \times 1.17 \text{ hr} = \mathbf{\$3.49 \text{ per 100 iterations}}$ - **Your machine is 20x cheaper** for compute!

5.3.2 Power Management

For long-running jobs, consider:

```
# Limit GPU power to reduce heat/noise
import subprocess

def set_gpu_power_limit(watts=400):
    """Limit RTX 4090 to 400W (vs. 450W default)."""
    for gpu_id in [0, 1]:
        subprocess.run([
            'nvidia-smi', '-i', str(gpu_id),
            '-pl', str(watts)
        ])
    print(f"GPU power limited to {watts}W")
    print("Performance impact: ~5-10% slower, much cooler/quieter")

# For overnight jobs where speed isn't critical
set_gpu_power_limit(watts=350) # 22% power reduction
```

Power limiting trade-offs: - 450W (default): Fastest, hottest ($\sim 80\text{-}85^\circ\text{C}$) - 400W: 5% slower, cooler ($\sim 75\text{-}80^\circ\text{C}$) - 350W: 10% slower, much cooler ($\sim 70\text{-}75^\circ\text{C}$), quieter

6 Cloud vs. Local Comparison

6.1 Lambda Labs Pricing

Instance	GPUs	VRAM	Cost/hr	100 iters	1000 iters
1x A100 (40GB)	1	40 GB	\$1.29	\$2.51	\$25.08
1x A100 (80GB)	1	80 GB	\$1.59	\$3.09	\$30.87
2x A100 (80GB)	2	160 GB	\$2.98	\$2.90	\$29.02
4x A100 (40GB)	4	160 GB	\$4.40	\$2.56	\$25.58

6.2 Your Local Machine

Metric	Your Hardware	Lambda (2x A100)
Setup cost	\$0 (own it)	\$0
Compute cost	\$0.17 / 100 iters	\$2.90 / 100 iters
Time (100 iters)	70 minutes	~75 minutes
Availability	Always	Rent by hour
Data transfer	None	Upload/download
Privacy	Local	Cloud
Flexibility	Full control	Limited

Break-even analysis: - Lambda: \$2.90 / 100 iters - Your machine: \$0.17 / 100 iters - **Savings:** \$2.73 / 100 iterations - For 10,000 iterations: Save \$273

6.3 When to Use Lambda Labs

Rent Lambda when: 1. Testing at massive scale (N=500, T=2000) - Needs >48 GB VRAM (your limit)
- 2x A100 (80GB) = 160 GB VRAM

- Running 10+ iterations simultaneously
 - You have 2 GPUs, Lambda can provide 4-8
- Your machine is needed for other work
 - Keep local machine free for daily tasks
- One-off massive experiments
 - 5,000+ iterations
 - Run over weekend: \$200-300 on Lambda vs. tying up your machine
- Testing code before deploying to your hardware
 - Cheap hourly rate for development

Use local machine when: 1. Regular research workflow (<200 iters/day) 2. Interactive development (frequent re-runs) 3. Data privacy required 4. Cost-sensitive work (student budget, unfunded) 5. Always-available compute needed

6.4 Hybrid Workflow Example

Typical research project (1600 iterations total):

Week 1-2: Development (Local) - Quick tests: 50 iterations - Cost: \$0.08 electricity - Time: ~35 minutes (dual GPU parallel)

Week 3: Baseline runs (Local) - 100 iterations baseline - Cost: \$0.17 electricity - Time: 70 minutes (dual GPU parallel)

Week 4: Sensitivity analysis (Decision point)

Option A - Local: - 1450 iterations overnight - Cost: \$2.47 electricity - Time: ~17 hours (runs while you sleep) - **Choose this if:** Machine can be dedicated overnight

Option B - Lambda (2x A100): - 1450 iterations - Cost: \$42 - Time: ~17 hours - **Choose this if:** Need your machine for other work

Recommendation: Use local for this example (saves \$40)

7 Production Deployment

7.1 Recommended Configuration Files

7.1.1 config_dev.yaml (Quick Testing)

```
# Fast testing configuration
model: bgn
N: 100
T: 720
num_iters: 5
n_jobs: 16          # Use all CPU cores
use_gpu: true
gpu_mode: "split"   # Fast single iterations
gpu_devices: [0, 1] # Both GPUs
output_dir: results/dev
```

7.1.2 config_batch.yaml (Production Runs)

```
# Large batch configuration
model: bgn
N: 100
T: 720
num_iters: 100
n_jobs: 8          # 8 cores per GPU task
use_gpu: true
gpu_mode: "parallel" # Dual GPU parallel iterations
gpu_devices: [0, 1]
output_dir: results/batch_$(date +%Y%m%d)

# Memory management
gpu_memory_fraction: 0.9 # Use 90% of GPU memory
clear_cache_every: 10    # Free memory every 10 iterations

# Monitoring
log_gpu_stats: true
log_interval_sec: 30
```

7.1.3 config_cloud.yaml (Lambda Labs)

```
# Cloud deployment configuration
model: bgn
N: 200          # Larger problem (requires 2x A100 80GB)
T: 1440
num_iters: 1000
n_jobs: 16
use_gpu: true
gpu_mode: "parallel"
gpu_devices: [0, 1] # Lambda 2x A100 instance
output_dir: /mnt/results

# Cloud-specific
```

```
checkpoint_every: 50    # Save progress every 50 iters
resume_from_checkpoint: true
```

7.2 CLI Interface

```
# main.py
import argparse

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--config', required=True)
    parser.add_argument('--gpu-mode', choices=['single', 'split', 'parallel', 'auto'])
    parser.add_argument('--num-iters', type=int)
    parser.add_argument('--gpu-devices', nargs='+', type=int, default=[0])

    args = parser.parse_args()
    config = load_config(args.config)

    # Override config with CLI args
    if args.gpu_mode:
        config.gpu_mode = args.gpu_mode
    if args.num_iters:
        config.num_iters = args.num_iters

    # Select processor based on GPU mode
    if config.gpu_mode == 'parallel':
        processor = DualGPUParallelProcessor(config)
    elif config.gpu_mode == 'split':
        processor = DualGPUSplitProcessor(config)
    else:
        processor = SingleGPUProcessor(config)

    # Run
    results = processor.run()

if __name__ == '__main__':
    main()
```

Usage examples:

```
# Quick test (5 iterations, split mode, 4 minutes)
python main.py --config config_dev.yaml --num-iters 5

# Medium batch (50 iterations, parallel mode, 35 minutes)
python main.py --config config_batch.yaml --num-iters 50 --gpu-mode parallel

# Large overnight job (500 iterations, parallel mode, ~6 hours)
nohup python main.py --config config_batch.yaml --num-iters 500 --gpu-mode parallel > batch.log 2>&1

# Cloud deployment
python main.py --config config_cloud.yaml --gpu-mode parallel
```

7.3 Monitoring Dashboard

```
# monitor.py - Real-time monitoring
import dash
from dash import dcc, html
import plotly.graph_objs as go
import subprocess
import pandas as pd

def get_gpu_stats():
    """Get current GPU stats."""
    result = subprocess.run([
        'nvidia-smi', '--query-gpu=timestamp,name,utilization.gpu,'
        'utilization.memory,temperature.gpu,power.draw',
        '--format=csv'
    ], capture_output=True, text=True)

    # Parse and return
    lines = result.stdout.strip().split('\n')[1:]
    stats = []
    for line in lines:
        parts = line.split(',')
        stats.append({
            'timestamp': parts[0],
            'gpu': parts[1],
            'util_gpu': float(parts[2].split()[0]),
            'util_mem': float(parts[3].split()[0]),
            'temp': float(parts[4]),
            'power': float(parts[5].split()[0])
        })
    return stats

# Create Dash app
app = dash.Dash(__name__)

app.layout = html.Div([
    html.H1('GPU Monitoring Dashboard'),
    dcc.Graph(id='gpu-utilization'),
    dcc.Graph(id='gpu-temperature'),
    dcc.Interval(id='interval', interval=2000) # Update every 2 seconds
])

@app.callback(
    [dash.dependencies.Output('gpu-utilization', 'figure'),
     dash.dependencies.Output('gpu-temperature', 'figure')],
    [dash.dependencies.Input('interval', 'n_intervals')]
)
def update_graphs(n):
    stats = get_gpu_stats()
    df = pd.DataFrame(stats)

    # Utilization graph
    util_fig = go.Figure()
```

```

for gpu in df['gpu'].unique():
    gpu_data = df[df['gpu'] == gpu]
    util_fig.add_trace(go.Scatter(
        y=gpu_data['util_gpu'],
        name=f'{gpu} - Compute',
        mode='lines'
    ))

# Temperature graph
temp_fig = go.Figure()
for gpu in df['gpu'].unique():
    gpu_data = df[df['gpu'] == gpu]
    temp_fig.add_trace(go.Scatter(
        y=gpu_data['temp'],
        name=f'{gpu} - Temp',
        mode='lines'
    ))

return util_fig, temp_fig

# Run in background during training
# python monitor.py &
# Open browser to http://localhost:8050

```

8 Troubleshooting Guide

8.1 Common Issues

8.1.1 Issue 1: CUDA Out of Memory

Symptom:

RuntimeError: CUDA out of memory. Tried to allocate 2.00 GiB

Diagnosis:

```
import cupy as cp

free, total = cp.cuda.Device(0).mem_info
print(f"GPU 0: {free/1e9:.1f} GB free / {total/1e9:.1f} GB total")

free, total = cp.cuda.Device(1).mem_info
print(f"GPU 1: {free/1e9:.1f} GB free / {total/1e9:.1f} GB total")
```

Solutions:

1. Free memory between iterations:

```
import cupy as cp

mempool = cp.get_default_memory_pool()
pinned_mempool = cp.get_default_pinned_memory_pool()

# After each iteration
mempool.free_all_blocks()
pinned_mempool.free_all_blocks()
```

2. Use float32 instead of float64:

```
# config.yaml
gpu_dtype: float32 # Half the memory usage

# In code
data_gpu = cp.asarray(data, dtype=cp.float32)
```

3. Reduce batch size:

```
# Process months in batches
batch_size = 180 # Instead of 360
for i in range(0, 360, batch_size):
    batch = months[i:i+batch_size]
    process_batch_gpu(batch)
```

8.1.2 Issue 2: Slow GPU Performance

Symptom: GPU code not much faster than CPU

Diagnosis:

```
# Check PCIe bandwidth
nvidia-smi --query-gpu=pcie.link.gen.current,pcie.link.width.current --format=csv

# Should be: 4, 16 (PCIe 4.0 x16)
# If you see: 4, 8 → GPU at half bandwidth!
# If you see: 3, 16 → GPU at PCIe 3.0 (older)
```

Solutions:

1. Check data transfer overhead:

```
import time
import cupy as cp
import numpy as np

# Measure transfer time
data_cpu = np.random.randn(1000, 1000)

start = time.time()
data_gpu = cp.asarray(data_cpu) # CPU → GPU
elapsed_transfer = time.time() - start

start = time.time()
result = cp.linalg.svd(data_gpu) # Computation
elapsed_compute = time.time() - start

print(f"Transfer: {elapsed_transfer*1000:.2f}ms")
print(f"Compute: {elapsed_compute*1000:.2f}ms")

if elapsed_transfer > elapsed_compute:
    print(" Transfer overhead dominates! Keep data on GPU longer.")
```

2. Profile with nvprof:

```
# Install CUDA toolkit
# Profile your script
nvprof --print-gpu-trace python main.py --config config.yaml --num-iters 1

# Look for:
# - % time in kernels (should be >80%)
# - Host-to-Device transfers (minimize these)
```

8.1.3 Issue 3: Both GPUs Running Same Iteration

Symptom: Both GPUs show same utilization pattern

Diagnosis:

```
# Check which GPU is being used
import cupy as cp

print(f"Current device: {cp.cuda.Device()}")
```

```
# In parallel mode, should see:
# Thread 1: Device 0
# Thread 2: Device 1
```

Solution:

```
# Explicitly set device in each thread
def process_on_gpu(iter_num, gpu_id):
    # IMPORTANT: Set device at start of thread
    cp.cuda.Device(gpu_id).use()

    # Verify
    assert cp.cuda.Device() == gpu_id

    # Continue with processing
    ...
```

8.1.4 Issue 4: Memory Leak (GPU Memory Grows)

Symptom: GPU memory usage increases over iterations

Diagnosis:

```
import cupy as cp

for i in range(100):
    # Run iteration
    process_iteration_gpu(i)

    # Check memory after each iteration
    free, total = cp.cuda.Device().mem_info
    used = total - free
    print(f"Iter {i}: {used/1e9:.2f} GB used")

    # If this number keeps growing → memory leak!
```

Solutions:

1. Explicit memory cleanup:

```
def process_iteration_safe(iter_num):
    try:
        result = process_iteration_gpu(iter_num)
    finally:
        # Always clean up, even if error
        mempool = cp.get_default_memory_pool()
        mempool.free_all_blocks()

    return result
```

2. Find the leak:

```

import cupy as cp
import gc

# Before iteration
cp.get_default_memory_pool().free_all_blocks()
gc.collect()
free_before, _ = cp.cuda.Device().mem_info

# Run iteration
process_iteration_gpu(i)

# After iteration
cp.get_default_memory_pool().free_all_blocks()
gc.collect()
free_after, _ = cp.cuda.Device().mem_info

leaked = (free_before - free_after) / 1e9
if leaked > 0.1: # >100 MB leaked
    print(f" Leaked {leaked:.2f} GB in iteration {i}")

```

9 Summary and Next Steps

9.1 Summary

9.1.1 Your Hardware

- **CPU:** Threadripper PRO 5955WX (16 cores) - Excellent
- **GPU:** 2x RTX 4090 (24 GB each) - Top-tier
- **Cost:** \$0 additional investment needed

9.1.2 Expected Performance

Configuration	Time/Iteration	Speedup	Development Time
Current (CPU)	30 min	1x	-
Phase 1 (Single GPU)	84 sec	21.8x	1 week
Phase 2 (Dual parallel)	84 sec*	21.8x	2 weeks
Phase 3 (Dual split)	47 sec	38.6x	3 weeks

*Throughput: 2x (43 iters/hour vs. 21.5)

9.1.3 ROI

Item	Value
Development cost	\$3,600 (60 hrs @ \$60/hr)
Annual savings	\$183,120 (3,052 hours)
ROI	51:1
Payback	7 days

9.2 Immediate Next Steps

9.2.1 Week 1: Setup and Planning

Day 1: Environment setup

```
# 1. Verify CUDA
nvidia-smi

# 2. Check PCIe bandwidth
nvidia-smi --query-gpu=pcie.link.gen.current,pcie.link.width.current --format=csv
# Should see: 4, 16 for both GPUs

# 3. Install CuPy
conda create -n gpu_accel python=3.10
conda activate gpu_accel
pip install cupy-cuda12x # Match your CUDA version

# 4. Test installation
python -c "import cupy as cp; print(cp.cuda.Device(0).name); print(cp.cuda.Device(1).name)"
```

Day 2-3: Code review and planning - Review IPCA code structure - Identify data dependencies - Plan CuPy conversion strategy

Day 4-5: Start IPCA GPU migration - Create `ipca_functions_gpu.py` - Implement `regress_gpu()` - Unit tests

9.2.2 Week 2: IPCA + SDF GPU

Day 1-3: Complete IPCA - Implement `ipca_iter_gpu()` - Implement `fit_ipca_gpu()` - Integration tests

Day 4-5: SDF GPU - Create `sdf_compute_gpu.py` - Convert matrix operations - Test numerical equivalence

9.2.3 Week 3: Dual GPU (Optional)

Day 1-3: Parallel iterations - Implement `DualGPUParallelProcessor` - Test with 10, 50, 100 iterations - Performance benchmarking

Day 4-5: Documentation and polish - Update README - Create usage examples - Performance report

9.3 Decision Points

9.3.1 After Week 1

Evaluate: Did single GPU Phase 1 work? - Yes, 20x+ speedup → Continue to Week 2 - No, issues → Debug before proceeding

9.3.2 After Week 2

Evaluate: Do you need dual GPU? - **Yes, if:** Running >50 iterations regularly - **No, if:** Mostly 1-10 iteration runs (single GPU sufficient)

9.3.3 After Week 3

Evaluate: Worth implementing dual GPU split? - **Yes, if:** >10 single-iteration runs per day - **No, if:** Mostly batch jobs (parallel is better)

9.4 Long-Term Strategy

9.4.1 Year 1: GPU Optimization

- **Q1:** Implement single GPU (Phase 1)
- **Q2:** Add dual GPU parallel (Phase 2)
- **Q3:** Optimize for larger N, T
- **Q4:** Consider Phase 3 if needed

9.4.2 Year 2: Scaling

- **Options:**
 - Keep current hardware (sufficient for most work)
 - Add cloud bursting for mega-batches (Lambda Labs)
 - Upgrade to 4x GPUs if running 10+ iterations continuously

9.4.3 Year 3+: Maintenance

- **Keep GPU code updated** with CuPy/CUDA versions
- **Monitor performance** - ensure no degradation
- **Consider alternatives** - JAX, PyTorch if ecosystem changes

9.5 Support and Resources

9.5.1 Getting Help

CuPy Documentation: - <https://docs.cupy.dev/> - <https://github.com/cupy/cupy/issues>

NVIDIA Developer Forums: - <https://forums.developer.nvidia.com/>

Stack Overflow: - Tag: [cupy], [cuda], [gpu-computing]

9.5.2 Benchmarking

Create baseline **benchmarks** before starting:

```
# Run 10 iterations on CPU
python main.py --config config.yaml --num-iters 10 --use-gpu false

# Save timing data
# Compare against GPU results after implementation
```

9.5.3 Success Metrics

Track these metrics to ensure GPU implementation is working:

1. **Speedup:** >20x for IPCA component
2. **Throughput:** >40 iterations/hour with dual GPU
3. **Accuracy:** Results match CPU within 1e-5 relative error
4. **Stability:** No memory leaks over 100+ iterations
5. **Utilization:** >80% GPU utilization during IPCA

9.6 Conclusion

You have **outstanding hardware** that's perfectly suited for GPU acceleration of your asset pricing simulations.

No additional hardware purchase needed - your 2x RTX 4090 setup is already better than most academic compute clusters for this specific workload.

Expected outcome: 21.8x speedup (30 min → 84 sec per iteration) with 1-2 weeks of development effort.

ROI: 51:1 - among the highest ROI projects you can undertake.

Recommendation: Start with Phase 1 (single GPU) this week. The benefits are enormous and the risk is minimal.

Ready to proceed? The next step is to set up CuPy and start migrating IPCA to GPU. Good luck!